LEITFADEN

ML im BIDA Team

Auf welchen Wegen wir Machine Learning und Deep Learning ausführen, warum es diese Wege gibt und wie du ein ML-Projekt startest.

Stand **21.10.2026** Wege **Snowsight · VS Code mit BIDA ML Starter** Ergebnisse **{env}_ML**

IN EINER MINUTE

Das Wichtigste auf den Punkt

1 **Zwei Wege**
ML läuft bei uns entweder direkt in Snowflake über Snowsight oder lokal in VS Code auf dem Notebook.

2 **Snowsight: nah an den Daten**
Die Daten bleiben in Snowflake. Du arbeitest mit den ML Functions per SQL oder in Snowflake Notebooks mit Python und musst nichts installieren.

3 **VS Code: volle Kontrolle**
Du arbeitest mit eigenen Modellen, der CUDA-GPU und Claude Code. Python kommt aus dem WinPython-Ordner, installiert wird nichts.

4 **Starter statt leerer Ordner**
In VS Code startet jedes Projekt aus dem BIDA ML Starter. Struktur, Snowflake-Anbindung und Leitplanken sind darin schon fertig.

5 **Ein Ziel für alle Ergebnisse**
Beide Wege schreiben in die ML-Datenbank `{env}_ML`. Ergebnistabellen und Views für den Betrieb werden in ITB modelliert. Modelle, Notebooks und Versuchstabellen entstehen in einem Arbeitsschema in `DEV_ML`.

DIE WEGE

Warum Snowsight, warum VS Code

Weg 1 · Snowsight

Warum: Rechnen, wo die Daten liegen. Du brauchst keinen Export und keine lokale Umgebung. So kommst du am schnellsten zu einem ersten Ergebnis.

ML Functions (SQL): Fertige Funktionen wie `FORECAST`, `ANOMALY_DETECTION` und `CLASSIFICATION`. Snowflake wählt Modell und Einstellungen selbst.

Snowflake Notebooks (Python): Python mit pandas, scikit-learn und Snowpark direkt in Snowflake. Die Pakete kommen aus dem Anaconda-Kanal von Snowflake. Einmalig muss jemand mit ORGADMIN-Rechten die Anaconda-Bedingungen akzeptieren. Pakete aus PyPI brauchen zusätzlich eine External Access Integration.

Weg 2 · VS Code

Warum: Wenn eine Standardfunktion nicht reicht. Du wählst Modelle, Features und Bibliotheken selbst, nutzt für Deep Learning die CUDA-GPU des Notebooks und lässt den Code von Claude Code schreiben.

Umgebung: Der WinPython-Ordner `C:\Users\<kürzel>\Documents\WPy64-313150` liefert Python 3.13 und PyTorch, ohne Installation und für alle gleich. Die GPU rechnet erst ab dem Stand mit PyTorch `cu130`.

Projekt: Geklont aus dem BIDA ML Starter, versioniert in Git.

Die Wege im Vergleich

	SNOWSIGHT	VS CODE MIT BIDA ML STARTER
Sprache	SQL (ML Functions) oder Python (Snowflake Notebooks)	Python in Jupyter-Notebooks
Wo gerechnet wird	In Snowflake, das kostet Credits	Lokal auf dem Notebook, mit CUDA-GPU
Daten	Bleiben in Snowflake	Aus Snowflake, vorerst per Export, bis die Anbindung steht
Modelle	ML Functions fest vorgegeben, in Notebooks frei mit den Snowflake-Paketen	Frei: Statistik, LightGBM, XGBoost, neuronale Netze
Deep Learning	Nur mit einem GPU-Compute-Pool. Das kostet Credits und braucht eine Freigabe	Ja, mit PyTorch auf der GPU
Unterstützung	Snowsight-Oberfläche	Claude Code mit Skills, Template-Notebooks, Prüfer-Agenten
Gut für	Schnelle Standardfälle, erste Ergebnisse, Vergleichswerte	Eigene Logik, mehrere Modelle im Vergleich, Deep Learning

Welcher Weg passt?

FRAGE 1 — **Löst eine ML Function die Aufgabe?** ▸ **Ja: Snowsight, SQL**

FRAGE 2 — **Reichen Python und die Snowflake-Pakete?** ▸ **Ja: Snowsight, Notebook**

SONST — **Eigene Modelle, GPU, Deep Learning** ▸ **VS Code mit BIDA ML Starter**

WARUM EIN STARTER-PROJEKT

Der BIDA ML Starter

Ohne Vorlage baut jedes Projekt seine eigene Struktur, eigene Verbindungen und eigene Regeln. Der BIDA ML Starter nimmt dir das ab. Du klonst ihn aus Git und kümmerst dich danach nur noch um deine Daten und dein Modell. Einzelheiten zu Ordnern, Templates und Claude-Dateien stehen in der Kurzanleitung zum BIDA ML Starter.

Einheitlich

Jedes Projekt ist gleich aufgebaut. Wer eines kennt, findet sich in allen zurecht und kann die anderen vertreten.

Reproduzierbar

Ein gemeinsamer WinPython-Stand, gepflegt von Stephan Meier, eine zentrale Config und ein fester Seed. Jeder Lauf lässt sich später genau nachstellen.

Kollaborativ

Code und Notebooks liegen in Git. Änderungen lassen sich teilen, vergleichen und reviewen.

Sofort startklar

Der Code für die Snowflake-Anbindung, das Schreiben nach `{env}_ML` und ein Smoke-Test für Umgebung und GPU sind schon da.

Geführt

Template-Notebooks für Forecasting, Klassifikation und Regression zeigen die Schritte. Den Code schreibt Claude Code mit den Skills.

Sicher

Hooks und Rules schützen Zugangsdaten und blockieren destruktives SQL, egal was Claude vorschlägt.

SO STARTEST DU

Ein ML-Projekt beginnen

Weg 1 · Snowsight

1. **Zugang prüfen**
   In Snowsight mit der Rolle `ML_DEVELOPER` und dem Warehouse `CONSUMER` anmelden.
2. **Daten finden**
   Die Quelle ist in der Regel eine View in `{env}_DATALAKE`, zum Beispiel eine PSA- oder `_90`-View.
3. **Werkzeug wählen**
   Ein SQL-Arbeitsblatt für die ML Functions oder ein Snowflake Notebook für Python. Das ML-Modell, zum Beispiel `CREATE SNOWFLAKE.ML.FORECAST`, und das Notebook sind Objekte und entstehen im Arbeitsschema. Dafür braucht `ML_DEVELOPER` eigene Rechte.
4. **Zielobjekte modellieren**
   Ergebnistabellen und Views in `{env}_ML` werden in ITB modelliert und deployt, nicht von Hand angelegt.
5. **Ergebnisse schreiben**
   Ergebnisse in die modellierten Tabellen schreiben, immer anhängen und mit Zeitstempel.

Weg 2 · VS Code

1. **Starter klonen**
   `git clone https://github.com/themanbetterknownasthemachine/BIDA-ML-Starter.git`, danach den Ordner in VS Code öffnen.
2. **WinPython wählen**
   Als Interpreter `…\WPy64-313150\python\python.exe` eintragen, im Notebook denselben Kernel wählen.
3. **Smoke-Test**
   `00_environment_check.ipynb` ausführen. Erwartet ist `GPU-Test: OK`. Die Snowflake-Zelle darf vorerst fehlschlagen.
4. **Daten anbinden**
   Die Quelle in `configs/config.yaml` eintragen. Solange die Anbindung fehlt, legst du einen Export in `data/raw/`.

5 Arbeits-Notebook anlegen

Zum Beispiel `work_<projekt>.ipynb`, daneben das passende Template öffnen. Pro Schritt fragst du Claude Code nach dem Code.

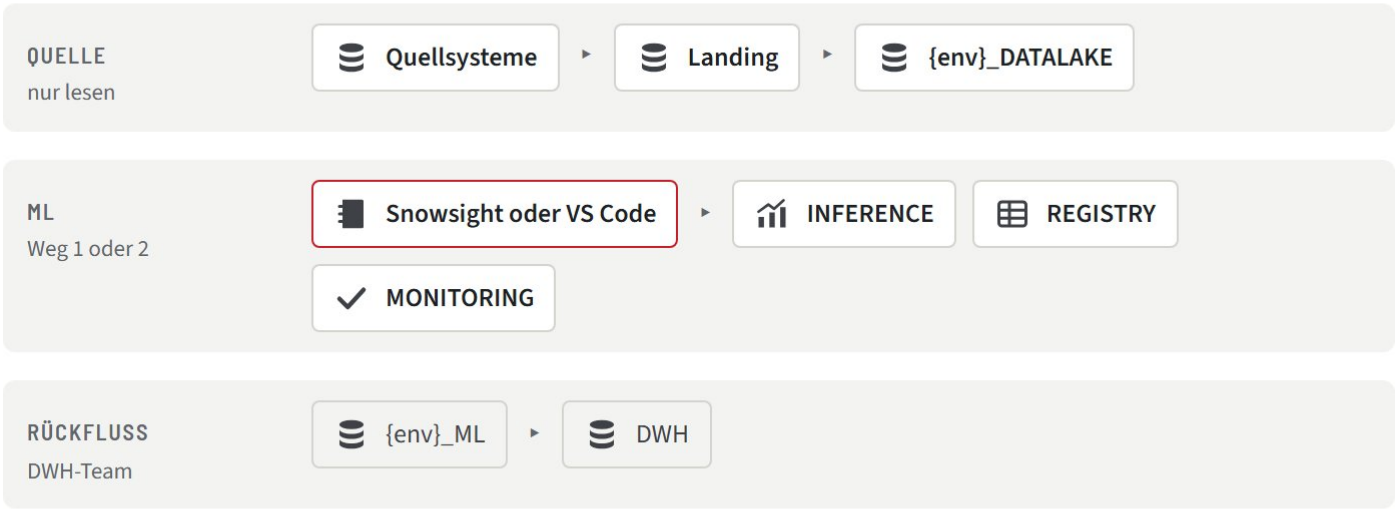
6 Ergebnisse schreiben

`write_to_snowflake()` hängt die Ergebnisse an Tabellen in `{env}_ML` an. Gibt es die Tabelle noch nicht, legt die Funktion sie selbst an. Das ist für Versuche in `DEV_ML` gedacht. Für den Betrieb wird die Tabelle in ITB modelliert.

> **Was wo entsteht:** Ergebnistabellen und Views für den Betrieb werden in ITB modelliert. Modelle, Notebooks und Versuchstabellen dürfen in einem Arbeitsschema in `DEV_ML` entstehen. Prozeduren und DDL für den Betrieb entstehen nie von Hand.

ARCHITEKTUR

Der Weg der Daten



Entwickelt wird in `DEV_ML` und getestet in `INT_ML`. Nach `PROD_ML` schreibt erst das offizielle Deployment. Den Weg der Ergebnisse im DWH legt das DWH-Team fest. Je nach Bedarf kommen dabei auch Staging, Raw Vault und Business Vault dazu.

BESTANDESAUFNAHME

Was brauche ich, was fehlt noch?

BAUSTEIN	WOFÜR	STATUS
Snowflake und Snowsight	Daten in `{env}_DATALAKE`, Rolle `ML_DEVELOPER`, Warehouse `CONSUMER`	VORHANDEN
ML-Datenbank `{env}_ML`	INFERENCE, REGISTRY, MONITORING. DEV_ML ist vorhanden, INT und PROD folgen mit dem Deployment	TEILWEISE
Notebooks mit CUDA-GPU	Lokales Training, auch für Deep Learning. Die GPU ist nutzbar ab dem WinPython-Stand mit PyTorch `cu130`	TEILWEISE
WinPython-Ordner	Python und Pakete ohne Installation	VORHANDEN
VS Code, Git, Claude Code	Entwickeln, versionieren, Code erzeugen	VORHANDEN
BIDA ML Starter	Projektvorlage für Weg 2	VORHANDEN
Snowflake aus VS Code	Die Connectoren sind im WinPython-Ordner enthalten. Die Anbindung ist aber noch nicht eingerichtet und nicht genehmigt	TEILWEISE
Paketpflege WinPython	Stephan Meier nimmt neue Pakete auf und verteilt neue Stände an alle	VORHANDEN

VORHANDEN nutzbar TEILWEISE mit Einschränkung